



Module 3 Day 1 Student Guide: Introduction to Linux

Class Overview

This week, you will be given an introduction to basic Linux systems and the command line. Additionally, you will learn about different distros, file structures, file navigation, and file manipulation.

Class Objectives

By the end of class, you should be able to do the following:

- Name three of the most important distributions of Linux
- Navigate the Linux file structure using the command line
- Discuss the prominence and utility of the command line in IT and security professions
- Use commands like `ls`, `cd`, `mkdir`, `touch`, `cp`, `mv`, `rm`, `rmdir`, `history`, `echo`, and `cat` for basic file navigation and manipulation
- Navigate deeply nested folder structures using relative and absolute file paths

01. So, Why Linux?

Welcome to “Introduction to Linux.” Today’s lesson will introduce the fundamental concepts and details of Linux systems.

While Linux is uncommon as an operating system on personal computers, it is widespread on the Internet and in enterprise networks. Millions of websites today are hosted with Linux operating systems.

The ubiquity of Linux machines on modern networks makes them a common target for attackers. Therefore, familiarity with this operating system is crucial for cybersecurity professionals.

This knowledge is essential for the following technical roles, among others:

- **Help desk/IT support:** Help desk and IT support specialists often need to troubleshoot issues on Linux workstations. People in these roles often find themselves troubleshooting issues with macOS as well. Since macOS shares most of its structure with "classical" Linux operating systems, this knowledge is valuable even for personnel who are not in charge of configuring or managing machines themselves.
- **Systems administration:** As most complex networks contain at least one Linux server, systems administrators need to understand how to audit, configure, and secure Linux servers and workstations.
- **Penetration testing:** Again, most complex networks contain at least one Linux server, making them frequent targets for pen testers. Since pen testers must exploit machines to gain access to them, deep knowledge of how the operating system works and where it stores its most important files is essential.
- **Network forensics:** Forensics specialists spend much of their time inspecting server logs. Often, these logs are generated by or stored on a Linux server, and forensic examiners must be able to navigate the file system. During investigations, specialists must be careful not to inadvertently alter the file system, which makes their knowledge of how and where data is stored on Linux machines a crucial element of the job.

Have you ever used Linux before?

Linux and artificial intelligence:

We learned about AI in the previous module. Linux plays a pivotal role in the AI landscape, as it provides a robust and flexible environment for developing and deploying AI technologies. It is the preferred platform for many AI frameworks like TensorFlow, PyTorch, and Keras, because it enables efficient creation and testing of AI models.

Most high-performance computing (HPC) clusters, essential for training complex AI models, run on Linux and benefit from its capable handling of massive datasets. The open-source nature of Linux aligns with the collaborative spirit of the AI community, fostering innovation and accelerating development. Its scalability makes it suitable for various applications, from small devices to large data centers, and its widespread use in cloud- and edge-computing environments facilitates the deployment and management of AI workloads.

Linux's security features and customization options ensure that AI solutions are both robust and secure. Overall, its integration with AI frameworks, support for HPC, and open-source ethos make Linux a cornerstone of AI advancement.

02. Linux History and Distributions

You may hear Unix and Linux used interchangeably, but what exactly is the difference?

- To begin understanding the differences, let's look at their history.

Brief history of Unix/Linux:

- In 1969–1970, Dennis Ritchie, Kenneth Thompson, and several others at AT&T Bell Labs began developing a small operating system, called **Unix**.
- Within the next two decades, different updates and distributions of Unix were created. Most were proprietary.
- In 1984, Richard Stallman's Free Software Foundation (FSF) started the GNU project, which created a free distribution of the Unix operating system.
- In 1991, Linus Torvalds used many of the components from the GNU project to create a free, open-source operating system, which is now known as **Linux**.
- In summary:
 - Unix came first.
 - Linux is "Unix-Like" and was derived from Unix.

What is an operating system?:

- An **operating system**, or OS, is the software that supports the basic functions of a computer, such as scheduling tasks, executing applications, and controlling peripherals.
- Windows, macOS, and Linux are all examples of operating systems.
- Of these operating systems, Linux is by far the most commonly used on the Internet, and most enterprise networks feature at least one Linux machine.

Unlike Windows and OS X, Linux is a **free, open-source software (FOSS)**.

- This means that anyone can read or modify its **source code**.
- Linux is free and open source specifically because Windows and Unix, Linux's early competitors, were not. In other words, Linux was developed for researchers and students in direct response to the fact that Windows and Unix were extremely expensive and/or inaccessible in the early days of computing.
- Today, many tools in the hacker/security community are still published as open source, and a belief in free software and free information is widespread in the industry. Almost all the tools students will use in class are open source, and nearly all technical security professionals are familiar with the Linux FOSS ecosystem.

Because Linux is a FOSS, many different people have developed their own special-purpose variants of the operating system. These variants are called **distributions**, or **distros**.

- There are many different distributions in use today. In this course, we will use two: **Ubuntu** and **Kali Linux**.
- Both Ubuntu and Kali Linux are distributions based on another distribution called **Debian**. Ubuntu is essentially Debian with customizations for general-purpose users, and Kali is Debian with customizations for security professionals.
- Since both distributions are based on Debian, everything you learn in this unit will transfer directly to all that you will learn about Kali in the latter half of the course.

One of the most important skills you will acquire in this unit is comfort with the **command line**, which you will use extensively on both Ubuntu and Kali.

- Knowledge of the command line is particularly important in the professional world, because most production Linux servers do not even offer a graphical interface. They can only be used from the command line!
- Command line-only machines are called **headless servers**. Headless servers are the norm because, by today's standards, the command line requires very few resources. This gives the server maximum resources to run its services and applications. A graphical user interface (GUI)-based system is neither required nor desirable.
- Being able to navigate a headless Linux server is the main reason we need to learn Linux and the Bash command line. You will be able to use the Bash command line environment to work with both Linux and Unix systems, such as macOS.

The next exercise will familiarize you with the most popular Debian and Red Hat distributions in use today. This will help you identify Linux machines on networks they meet "in the wild" and determine which distributions to install when setting up networks on the job.

03. Activity: Distribution Research

In this exercise, you will play the role of a **system administrator (sysadmin)** at X Corp, a company that has recently experienced a number of breaches involving servers running outdated Linux distributions. In response, the IT Department has decided to upgrade the affected servers with newer distributions and versions of Linux. You will be responsible for conducting research to determine which distribution is most appropriate for each machine.

You can access the file using the following link:

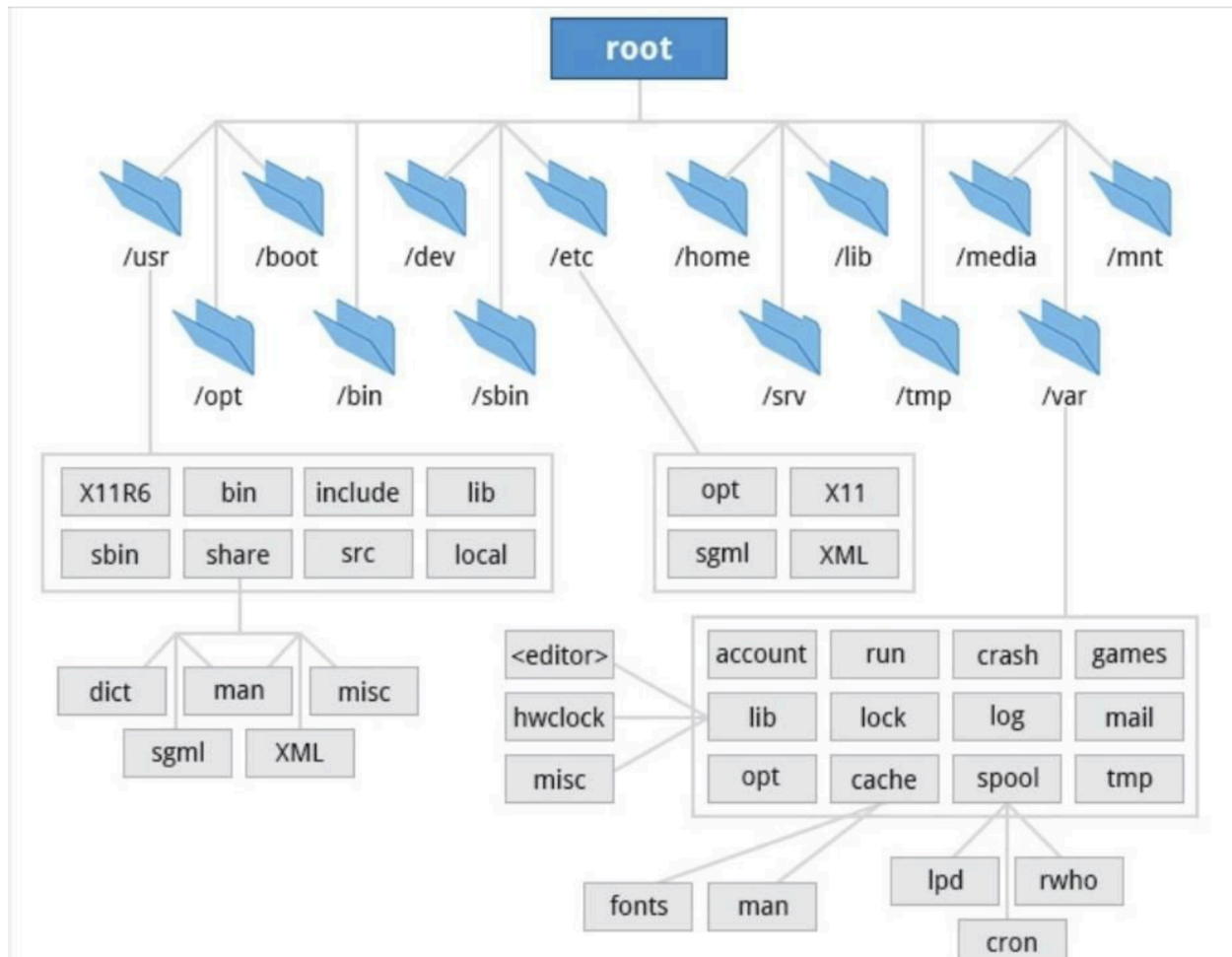
- [Activity File: Distribution Research](#)

While there are hundreds of distributions in use, this exercise will focus only on those you are likely to encounter on the job.

04. Review: Distribution Research

The instructor will review the activity with the class.

05. Linux File System Structure



- All operating systems maintain certain conventions for where and how they store different kinds of files. For example, Windows and macOS machines typically place images in a `~/Pictures` folder.
- Linux also has a conventional file system structure.
- These conventions are consistent across distributions. For example, user applications are typically stored in `/usr/bin`, and this will generally be the case across distros.
- Therefore, you can apply what we will learn today to other distributions you encounter, as they will likely all have the same files and folders.
- In addition, most Linux distributions use the Bash shell by default. This means you can use the command line tools, which you will learn here, on other distros as well.
- Being able to use the same command line on many distributions is a major advantage, because GUI-based desktop environments are not usually the same across distros.

- For example, Ubuntu and Fedora look very different and need to be configured with different GUI tools, but the shell commands used to configure each machine are almost identical.
- Understanding the command line allows administrators to work with any Linux machine using a Bash shell, even if it has an unfamiliar desktop environment.

This means you will be able to use the skills you learn today to navigate any Linux server you come across.

Once you master the structure of the Linux file system, you can better identify potentially malicious applications and files where they do not belong. Often, this indicates that the system has been compromised.

- Before we look at the Linux file structure, it is important to know that the term “root” can be used to refer to multiple contexts.
 - Root may refer to any of the following
 - The user account for the administrator
 - The top-level directory that all directories fall under
 - The home directory for the **root** user account
- We will further explore these in this section.

There are several conventional directories in Linux:

- **/:** This is the **root** directory, which contains every other directory on the system.
- **/home:** This directory contains each user's private files. Users should not be able to save files anywhere else.
- **/root:** This directory is the home directory for the **root** user. This can only be accessed with root privileges.
- **/etc:** This directory contains configuration files that define how the machine should run and who is allowed to use it. For example, this directory includes information on registered users and their passwords.
- **/bin and /sbin:** These directories contain applications such as web browsers, and commands such as `ls`.
- **/var:** This directory contains files that change over time. For example, `/var/log` contains log files, which programs update frequently as they run.
- **/tmp:** This directory contains files that are only needed for a short period of time.

There are other important directories, such as `/proc` and `/boot`. However, these have very specific use cases that are beyond the scope of today's lesson.

Tour of the Linux file structure

Let's go through the Linux file system:

/ Root Directory

“/” refers to the root directory—that is, the directory that contains every other directory. All files and folders are ultimately contained within the root directory.

- Run: `cd /`
- Run: `ls`

Your output should resemble the following:

```
Unset
$ cd /
$ ls
bin boot cdrom dev etc home initrd.img initrd.img.old lib
lb32 lib64 lost+found media mnt opt proc root run sbin
snap srv swapfile sys tmp usr var vmlinuz
```

Each item listed here is a directory, because the main purpose of `/` is to store other crucial directories, not bare files.

- Because we are working within a VM, we might have mounting provisioners such as `initrd.img`, `initrd.img.old`, `swapfile`, and `vmlinuz`. Although we won't cover these in great detail, these files allow the Ubuntu virtual machine to communicate with our host machine.

The next directory we'll explore is `/home`, where users store their own private data.

/home Directory:

- Linux allows you to create multiple user accounts. This allows many people to use the machine at the same time.
- Each user is allowed to personalize their environment. For example, users can change their desktop backgrounds or install new software.
- The files that contain each user's personal preferences are called **local configurations**, or **configs**. These configurations only apply to the user who wrote them.
- Each user is given their own dedicated folder on the file system. This is where they store their own private files, folders, and local configurations.
- By default, users are only allowed to modify files and folders in their own dedicated directory. This means they cannot modify files in other users' directories, or in protected folders like `/`.
- These private directories are stored in `/home`.

The name of a user's private directory is the same as their username.

- For example, the user `sally` would have a directory at `/home/sally`, while the user `jack` would have a directory at `/home/jack`.
- Only active registered users should have private directories in `/home`. Directories belonging to users who have been deleted or who no longer use the computer should be deleted.

- **Note:** `root` is a special user, and `root`'s home directory is not `/home/root`, but rather `/root`.

/etc Directory

- The `/etc` contains configuration files that define how the machine should run and who is allowed to use it.
- While normal users can only modify their own local configurations, an administrator can make changes that apply to all users of the system.
- These are called **system-wide configurations**, and apply to every user on the system. For example, an administrator might configure a machine to prevent people from going to <https://facebook.com>.
- Configurations are typically stored as simple text files. Therefore, they can easily be inspected with familiar tools like `cat` and `less`.
- The files `/etc/passwd` and `/etc/shadow` contain information about all registered users on the system.
- `/etc/passwd` contains a list of registered users on the system. Contrary to its name, it contains no information about user passwords.
- This means that attackers who steal `/etc/passwd` will get a list of users to attack, but will not be able to steal their passwords.
- By contrast, `/etc/shadow` contains both a list of usernames and information about their passwords.
- Specifically, it stores something called a **password hash**. This is a string that is different from the user's password, but the operating system can still use it to check that they've entered the correct password.
- Because hashes aren't passwords, they cannot be used to log in. Additionally, the process of turning a hash back into a password, called **password cracking**, is extremely time-consuming.
- This means that an attacker who steals `/etc/shadow` cannot break into user accounts without first spending the time to crack their passwords.
- In general, an attacker cannot crack strong passwords, but cracking passwords is much easier to do if the passwords are weak.
- Since most users do not use strong passwords, an attacker who steals `/etc/shadow` is likely to be able to crack at least one of the passwords it contains.
- This makes `/etc/shadow` one of the **most important files** to protect on a Linux system.
- Run `less /etc/passwd` to show the contents of `/etc/passwd`.
- Run `sudo less /etc/shadow` to show the contents of `/etc/shadow`.
 - We need `sudo` to read `/etc/shadow` because the permissions on that file are strict. `sudo` temporarily grants `root` permissions.
- The `/etc/shadow` file appears nearly identical to `/etc/passwd`, with one stark difference: instead of an `x`, the second field contains a long, complex string, such as `$6$6Y/fI1nx$zQJj6AH9asTNfhxV7NoVgxByJyE.rVKK6tKXiOGNCfWBsrTGY7wtC6Ce p6co9eVNkRFRpK6koXs1NU3AZQF8v/`.
 - This string is a password hash.

- This hash cannot be used to log in, but can be cracked to retrieve the user's original password.
- If a row in `/etc/shadow` contains a `!` instead of a password hash, this means the account is locked.
- If a row in `/etc/shadow` contains a `*` instead of a password hash, this means the user is not allowed to login.
 - This is usually implemented on a user account that is reserved for the system, and not a human user.

/sbin and /bin Directories

Now, we will look into several directories that contain program files that run on your Linux distribution.

- Every application or command you run on a Linux system is a program whose code is stored in files.
- Files containing programs are unique because you can run them. This is in contrast to other "normal" files, such as PDFs, which you can only read and write.
- Files you can run are called "executable files" because they execute some task.
- All software, including malware, is stored in executable files.
- Malware files can be dangerous because there is nothing preventing them from doing things that might harm the system.
- Executables can be stored anywhere on the file system, but the two main directories that store commands are `/sbin` and `/bin`.
- `bin` stands for "binary," because executable files contain "machine code." This is code written in 1s and 0s that is not readable by humans.
- The `/bin` directory contains programs used by average users, such as `ls` and `cp`.
- The `/sbin` directory contains programs used by administrators, such as `adduser`, which can add users to a system, and `passwd`, which is used to change a user's password.
- Run `ls /bin` to display the contents of `/bin`.
- Run `ls /sbin` to display the contents of `/sbin`.

Output should resemble the following:

```
Unset
$ ls /bin
bash          cp            grep          ls
ntfs-3g.probe ps            systemd
unicode_start
bunzip2       cpio          gunzip        lsblk         ntfscat
pwd           systemd-ask-password vdir
busybox       dash          gzexe         lsmod
ntfsccluster  rbash        systemd-
```

...

Display the contents of `/sbin` by running `ls /sbin`:

Unset

```
$ ls /sbin
```

acpi_available	e2image	ifdown	lvdisplay
mkfs.vfat	pvresize	unix_chkpwd	
agetty	e2label	ifquery	lvextend
mkhomedir_helper	pvs	unix_update	
apm_available	e2undo	ifup	lvm
mkntfs			

/var Directory

Many programs save information about errors or suspicious behavior in **log files**. These log files often contain valuable evidence about malicious activity.

If the list of installed programs seems fine, you will proceed to verify that the programs are saving the expected logs.

- Log files change constantly, because programs that keep logs add information to them as they run. This means that these files are **variable**; in other words, they change over time.
- Files that change over time are stored in `/var`, for "variable."
- Log files are stored in their own directory, called `/var/log`.
- Other types of variable files are stored in `/var`, but we will not discuss them in today's class.
- Linux systems may contain the following logs:
 - `/var/log/ufw.log`: This stores firewall logs, including attempts by users to visit unauthorized sites.
 - `/var/log/auth.log`: This stores failed login attempts when a user has tried to log in as the root user.
- Run `ls /var/log` to display the directory's contents. You should see some of the following logs:

Unset

```
$ ls /var/log
```

```
alternatives.log  btmp  kern.log  syslog  apport.log  dpkg.log
auth.log
```

- You should not have a `ufw.log`.
- The above listing reveals that the server is not maintaining firewall logs, although it is saving records about failed login attempts.
- This means that you do not know if users have visited malicious websites, but you can tell if anyone has tried and failed to log in as an administrator or different user.

/tmp Directory

We have one last directory to explore: `/tmp` stores temporary files and is often attacked by cybercriminals.

- Logs are meant to be permanent. In other words, programs expect to add information to log files, and not rewrite or delete them and create new ones (though administrators can do these things manually).
- Programs often need to create files that do not need to be permanent. For example, a text editor might save copies of your work every few minutes, even if you do not manually save it, to ensure it can restore your work if you accidentally close the editor before saving.
- Such files are variable in that they change regularly, but also temporary in that they do not need to be maintained long term.
- Since these files are different from logs and other such files, they are typically stored in their own dedicated directory: `/tmp`.
- Since files saved to `/tmp` are intended as temporary, the operating system deletes files in `/tmp` whenever the machine is restarted.

Because files in `/tmp` are deleted between reboots, cybercriminals often save malicious files or scripts to this directory knowing that they will be available for use while they are exploiting the machine, but automatically deleted *by the target itself* at reboot.

Summary:

- `/etc` stores host-specific system-wide configuration files, as well as the most sensitive files on a Linux system.
- `/var` stores files that are continually updated.
- `/home` stores users' home folders.
- `/tmp` is the directory that applications write temporary files, which can be deleted on reboot, to.
- `/bin` and `/sbin` are where the system keeps its main *binary* or program files.

The next exercise will give you an opportunity to explore the directory structure within your Linux VM.

06. Why Terminal Matters

You are likely familiar with a Microsoft Windows or macOS interface

- These interfaces are called **graphical user interfaces** or **GUIs** (pronounced "gooey").
- They are called GUIs because they contain graphical elements, such as icons, windows, and buttons.

Within these macOS and Windows GUIs, you can accomplish many user tasks by directly clicking your mouse on the screen, including the following:

- Creating files
- Viewing files
- Moving files
- Searching through directories

We will now learn how to use a different interface, called the **command line**, to complete these tasks.

Have you ever used the command line before?

What is the command line?

- Unlike the Windows and macOS interfaces, command line users do not directly interact with icons and buttons on the screen.
- Rather, users complete tasks on their machine by issuing commands with a line or lines of text.
 - For example, rather than clicking on a file to open it, the user types a command that opens the file.
- The command line is a critical core competency for IT and security professionals.

Why the command line?

While the command line may seem challenging at first, learning and using it provides important benefits for IT and security professionals. The command line can be:

- **The only way to achieve a desired outcome**
 - In many cybersecurity and IT jobs, you will find yourself working with systems and tools that have no GUI interface. This is typical of many servers.
 - For example, as a security professional, you may need to configure a system that does not have a GUI, and the command line will be your only mode for configuration.
- **The fastest way to achieve a desired outcome**

- The command line has many tools and scripts that are not available on a GUI. These tools can often speed up a task.
- For example, you can develop a script on a command line that can automate and repeat a task.
- **The most flexible way to achieve a desired outcome**
 - While working with a GUI may be the most familiar, working directly with the command line provides greater control.
 - For example, you can output logs to exactly where you want, combine files in unique ways, and string commands together with more flexibility.

The best way to learn the command line is to use it. Therefore, this module is built around hands-on exercises.

In today's exercises, you will play the role of a security analyst at a candy manufacturing company, Candy Corp:

- You will be using the command line to investigate a potential rogue employee, who may be selling secret Candy Corp recipes to a rival organization, Sugar Corp.
- You will be tasked with using the command line to identify evidence of suspicious activity from this rogue employee that has been left on the system.

07. Basic Terminal Commands

Within Windows Explorer, you can complete the following common tasks:

- View your current location by viewing the current directory path at the top of the Explorer page.
- Navigate through the directories and files by clicking directly on the folder to access its subfolders or files.
- Create folders by right-clicking and selecting **New** → **Folder**.

We will be completing these same tasks on the command line using the following text commands:

- `pwd` displays the current working directory
- `ls` lists the directories and files in the current directory
- `cd` navigates into a directory
- `mkdir` makes a directory
- `rmdir` removes a directory
- `touch` creates an empty file
- `rm` removes a file
- `clear` clears the terminal history on the page
- `echo` displays (or prints) text on the terminal
- `history` displays the commands you previously ran

Imagine the following scenario:

- You are a security analyst at ACME Corp and your manager has assigned you several security investigations.
- They provided you with access to a computer in the evidence room and need you to do the following:
 1. Create several directories to organize your investigations: `Case1` and `Case2`.
 2. Put your directories in the existing folder `security_evidence`.
 3. Put an empty file in the `Case1` directory titled `case1_evidence`. Later, you will use this file to store evidence.

This computer doesn't have a GUI, so we have to complete all these tasks on the command line.

Basic terminal demonstration

Watch as the instructor does the following:

1. Open an instance of a terminal on your Ubuntu virtual machine (VM0).
 - For our scenario, this is the new computer.
2. Next, let's run our very first terminal command using `echo`.
 - `echo` is used to simply display or print out text on the command line.
 - Run `echo "Hello World"` and press Enter.
 - This will display "Hello World" back on the terminal.
3. Now that we have completed our first terminal command, let's begin our manager's request by figuring out the current location in the file directory.
 - To find your current location, run the `pwd` command to display the path.
 - `pwd` stands for "print working directory" and this command displays your current location.
 - Our current location should display as `/03-instructor/day1/terminal_demonstration`.
4. Now that we know where we are located, we need to know which other folders we can access and navigate into.
 - Run the `ls` command to see all the files and folders in your current directory.
 - `ls` provides a list of files and folders in the current folder.
 - The results only display a directory called `security_evidence`.
5. Next, we will navigate to the `security_evidence` folder, as our manager requested.
 - Enter in the command `cd security_evidence`, but don't run it yet.
 - Explain that `cd` stands for "change directory" and that this will navigate into the `security_evidence` directory.
 - The syntax is `cd <folder name>`.
 - Delete `cd security_evidence`, and only type `cd sec`.
 - Push the Tab button and point out that pressing Tab will autocomplete the directory based on the first few letters you selected and the same `cd security_evidence` will display.
 - Press Enter to run the command.

- Run the `pwd` command and notice that you are now located in the `/03-instructor/day1/terminal_demonstration/security_evidence` folder.
6. We are now in the `security_evidence` folder and can create several directories in which to place our evidence.
- Run the command `mkdir Case1`.
 - `mkdir` stands for "make directory."
 - The syntax is `mkdir <folder name>`.
 - Run the `ls` command to confirm that the directory of `Case1` has been created.
 - Run the command `mkdir Case2`.
 - Run the `ls` command again and see that the directories for `Case1` and `Case2` have been created.
7. Security analysts often make mistakes when creating directories. Fortunately, there is a command to undo these mistakes.
- Run the command `mkdir Case3`. Remember that in this scenario, you accidentally created a directory that was not requested by the manager.
 - You can then use the `rmdir` command, which stands for "remove directory" and will remove a directory as long as it has no contents.
 - The syntax is `rmdir <folder name>`.
 - Run the `rmdir` command and then run `ls` to confirm that `Case3` no longer exists.
8. Our manager's last request was to put an empty file called `case1_evidence` in the `Case1` directory.
- First, run `cd Case1` to navigate into that folder.
 - Run the following command: `touch case1_evidence`.
 - Explain that `touch` is a command that creates an empty file.
 - The syntax is `touch <file name>`.
 - Run `ls` to confirm that `case1_evidence` has been created.
 - Explain that if you accidentally create the wrong file, similar to `rmdir`, you can use `rm <file name>` to remove a file.
9. You have now completed your manager's requests and can use the `clear` command to clear the terminal history on your page.
- The `clear` command removes the text from the page and starts from a fresh command line.
10. Lastly, if you want to see the commands you previously ran, run the following command: `history`.
- Run the `history` command to see how it shows a history of every command you have entered in the terminal.

Summary:

- `pwd` displays the current working directory.
- `ls` lists the directories and files in that current directory.
- `cd` navigates into a directory.
- `mkdir` makes a directory.

- `rmdir` removes a directory.
- `touch` creates an empty file.
- `rm` removes a file.
- `clear` clears the terminal history on the page.
- `echo` displays text on the terminal.
- `history` displays a list of the commands you previously ran.
- `tab` can be used to autocomplete your commands.

08. Activity: Take Five and Practice the Command Line

- [Activity File: Take Five and Practice the Command Line](#)
-

09. Review: Take Five and Practice the Command Line

The instructor will review the activity with the class.

10. Relative vs. Absolute Paths

We have just covered several commands that assist with file navigation and creation.

The unique location in the directory where a file is created is called a **file path**.

- A file path is like a physical path, as you have to follow a specific route to reach your destination.

Use the following hypothetical example:

```
/home/Sally/Desktop/my_images/cat.jpg
```

- The file is an image called `cat.jpg`.
- The directory path is `/home/Sally/Desktop/my_images/`.
- The `cat.jpg` image file is located in the `my_images` directory, which is in the `Desktop` directory, which is in the `Sally` directory, which is in the `home` directory.

File paths are often used by IT and security professionals to describe where files and other directories are loaded.

- For example, if you are a security administrator and need to access a network log so that you can conduct an investigation, you may need to ask the network team for the location or path of a network log file. The network team would likely provide you with a path, such as `/var/log/logfile1.txt`.
- This means the `logfile1.txt` log file is in the `log` directory, which is in the `var` directory.

There are two methods for providing paths to a file or directory on the command line: **absolute paths** and **relative paths**.

- An absolute path indicates the location of a file ,regardless of the user's current location in the file directory.
- In other words, if you are in the `Applications` directory and need to access a file in the `Desktop` directory, you can use an absolute path to get that file without first navigating to the `Desktop` directory.
- When writing an absolute path, include the top of the directory structure:
For example, we want to access a file in
`/home/Sally/Desktop/Sallysfiles/textfiles/Sallyfile.txt`.
 - If we are located in another directory, such as `/home/bob/file_logs/`, you can use an absolute path of
`/home/Sally/Desktop/Sallysfiles/textfiles/Sallyfile.txt` to access `Sallyfile.txt`.

A **relative path**, on the other hand, starts at the user's current location in the file directory. It doesn't require you to start from the top of the directory structure.

- In other words, if you need to access a file in the `Desktop` directory and you are already in the `Desktop` directory, you can create a path starting from your current location.
 - For example, to access `Sallyfile.txt`, you can use
`Sallysfiles/textfiles/Sallyfile.txt`.
 - You are in the `Desktop` directory, so `home/Sally/Desktop/` is assumed and is not written.
- For relative paths, you can also use `./` to indicate a relative path from the current directory. This is known as a **dotslash**.
 - The `.` indicates the current directory, and the `/` is the start of the relative path.
 - For example, to access `Sallyfile.txt`, you can use
`./Sallysfiles/textfiles/Sallyfile.txt`.

File paths on the command line

Relative and absolute paths are very helpful shortcuts when working on the command line.

- For example, let's say you want to access the `Sallysfiles` directory from the previous example, starting at the `home/Sally` directory.
 - Instead of running the following three commands from the `home/Sally` directory:
 - `cd Desktop`
 - `cd Sallysfiles`
 - `cd textfiles`
 - You can use the relative path from the `home/Sally` directory as a single command:
`cd Desktop/Sallysfiles/textfiles/.`

Path navigation demonstration

Watch the instructor go through the following steps:

1. Similar to the last demonstration, you need to know your current location in the file directory.
 - To find out your current location, run the `pwd` command to retrieve the path.
 - Your current location should display as
`/03-instructor/day1/pathnav_demonstration/`.
2. Next, navigate to the `security_evidence/Case2/` directory.
 - To navigate to that directory using a relative path, run:
`cd security_evidence/Case2/`
 - You have to be in the `pathnav_demonstration` directory to use this relative path.
 - Run `pwd`. You should now be in the
`/03-instructor/day1/pathnav_demonstration/security_evidence/Case2/` directory.
 - Create an empty file called `case2_evidence` by running `touch case2_evidence`.
 - Run `ls` to confirm the file has been created.
3. Next, navigate into the directory `Case1`, as your manager requested.
 - To navigate into that directory using an absolute path, run: `cd /03-instructor/day1/pathnav_demonstration/security_evidence/Case1/`.
 - No matter where in the file system you are, this absolute path will take you to the `Case1` directory.
 - Run `pwd` again. You should now be in the
`/03-instructor/day1/pathnav_demonstration/security_evidence/Case1/` directory.
 - Create an empty file called `Web_logs` by running `touch Web_logs`.
 - Run `ls` to confirm the file has been created.
4. Lastly, you have been asked to return to the `home` directory.
 - Explain that, in order to go backwards out of a directory, we use the `cd ../` command.
 - To go out of multiple directories, we simply add a `../` for each directory.
 - For example, to go backwards out of four directories, enter the command `cd ../../../../`.
 - To go back to the `03-instructor` from our current location, we need to go back four directories.
 - Run the command `cd ../../../../`.
 - Run `pwd` again to demonstrate that you should now be back in the `/03-instructor/` directory.

Copying and moving files

There are several more important basic commands, similar to the `cd` command, that we can use.

Security and IT professionals are often asked to **copy** files from one location to another.

- For example, a security analyst may be asked to make a copy of a log file and place the copy in a separate evidence folder. This way, if notes need to be added to the copied file, the original will remain untouched.
- The command `cp`, which stands for "copy," creates a copy of the file and places it in a specified location.
- The original file will remain, and a copy of the original will be moved to the new location.

Security professionals are also often asked to **move** files from one location to another.

- For example, an image file may have been accidentally placed in the email directory. An IT professional may need to move the file out of the email directory and into the correct image directory.
- The command `mv`, which stands for "move," moves the file from the current location into a new, specified location.
- Unlike copy, the original file will not remain, and the file will be moved into the new location.
- Note that `mv` is similar to cut and paste commands.

The syntax for both `cp` and `mv` is as follows:

```
<command> <file> <location to copy or move>
```

- In the example, `mv cat.jpg /dirA`, the command will move the file `cat.jpg` into the directory `dirA`.

The `<file>` or `<location>` can be indicated with an absolute or relative path if the file is not in your current location.

- For example: `mv dirA/dirB/dirC/cat.jpg /dirA/dirB`
 - This command will move the file `cat.jpg` from the `dirC` directory to the `dirB` directory.

Copying and moving demonstration setup

- Your manager realized they made a mistake. The web activity they want you to gather is only for Case 2. Therefore, you need to move the `Web_logs` file from the `Case1` directory to the `Case2` directory.
- After you move the file into the `Case2` directory, make a copy of the file `Web_logs` in the `security_evidence` folder. You will be able to add notes to this file later.

Copying and moving files demonstration

Watch your instructor go through the following steps:

1. Similar to the previous demonstrations, you need to know your current location in the file directory to get started. Run the `pwd` command to retrieve the path of your current location.
 - Your current location should display as
`/03-instructor/day1/pathnav_demonstration.`
 - Change back to `/03-instructor/day1/pathnav_demonstration` if you are not currently there.
2. Your manager has asked you to move the file called `Web_logs` from `Case1` to `Case2`.
 - We will use the move command.
 - Run the following:

```
mv
/03-instructor/day1/pathnav_demonstration/security_evidence/Case1/Web_logs
/03-instructor/day1/pathnav_demonstration/security_evidence/Case2/
```

 - This command will move the `Web_logs` file from the `Case1` directory into the `Case2` directory.
 - Note that we are using absolute paths for the file and the new destination.
3. Your manager asked you to make a copy of the file `Web_logs` from `Case2` in the `security_evidence` folder.
 - We will use the copy command.
 - Run the following:

```
cp
/03-instructor/day1/pathnav_demonstration/security_evidence/Case2/Web_logs
/03-instructor/day1/pathnav_demonstration/security_evidence/
```

 - This will make a copy of the `Web_logs` file from the `Case2` directory in the `security_evidence` directory.
 - Note that, again, we are using absolute paths for the file and the new destination.
4. Lastly, we should check to confirm our copy and move commands worked.
 - First, we will `cd` into the `/security_evidence` directory.
 - Run `ls` and confirm that the copy of the `Web_logs` file is in this directory.
 - Next, run `cd Case1` in the `Case1` directory and confirm the `Web_logs` file has been moved out of this directory.
 - Then, by using an absolute path, change into the `Case2` directory by running:

```
cd
/03-instructor/day1/pathnav_demonstration/security_evidence/Case2/.
```
 - Confirm the `Web_logs` file has been copied to this directory by running `ls`.

Summary:

- **File path** identifies a unique location in a file system.
- **Absolute path** is a path that includes the top directory of the file system.
- **Relative path** is a path that begins from the current directory.
- `cp` copies files.

- `mv` moves files.

In the next activity, you will use these same concepts to continue preparing for an investigation into the malicious employee who may be selling secret recipes to Candy Corp's rival organization, Sugar Corp.

11. Activity: Find your Milky Way (0:20)

- [Activity File: Find your Milky Way](#)
-

12. Review: Find your Milky Way (0:10)

The instructor will go over the solution guide in class:

13. Summary

- The secret to success in Linux is practice. Linux will become easier over time. Just think about how much you have learned in a single day.